



TROUBLESHOOTING HANDBOOK

VIRTUAL CONTROL
VMWARE CLOUD FOUNDATION SOLUTIONS

VMware vSAN and VCF Troubleshooting Handbook

vSAN cluster partition diagnosis, vSwitch failover policy resolution, and vCenter VM recovery procedures for VCF lab environments.

VSAN

VCF

CLUSTER PARTITION

RECOVERY

VCF 9.0

VMware Cloud Foundation

VMware vSAN and VCF Troubleshooting Handbook

PowerShell users: Most bash blocks in this document have a paste-ready PowerShell sibling block underneath (where translation is meaningful). vSAN-specific blocks translate to PowerCLI (`Connect-VIServer` , `Get-VsanDisk` , `Get-VsanResyncingComponent` , `Get-EsxCLI` for the `vsan` namespace). For the full substitution table — `curl` → `curl.exe` , `2>/dev/null` → `2>$null` , `here-docs` → `here-strings` , `jq` → `ConvertFrom-Json` , etc. — see [POWERSHELL-TRANSLATION-REFERENCE.md](#). Blocks that target the ESXi host shell only (`cmmms-tool` , `objtool` , `vsish` , `vmkfstools` , `partedUtil` , `vmkping` , and low-level `esxcli vsan debug ...` commands without clean PowerCLI parity) are flagged "Run on ESXi host shell only" with no PS sibling.

Table of Contents

Environment Information

- [Environment Details](#)
- [Network Configuration](#)

Problem Overview

- [Issue: vSAN Cluster Partition](#)
- [Symptoms](#)

Troubleshooting Procedures

- [Step 1: Verify vSAN Cluster Status](#)
- [Step 2: Verify VMkernel Adapter IP Configuration](#)
- [Step 3: Verify Physical NIC Status](#)
- [Step 4: Verify vSwitch Configuration](#)
- [Step 5: Check vSwitch Uplink Assignment](#)
- [Step 6: Check Failover Policy \(Critical\)](#)
- [Step 7: Verify VMkernel Port Configuration](#)
- [Step 8: Test vSAN Network Connectivity](#)
- [Step 9: Verify Routing](#)
- [Step 10: Check Security Policies](#)

vCenter Recovery

- [vCenter VM Recovery Procedures](#)
- [Register VM from Datastore](#)
- [VM Power Operations](#)
- [VM Configuration](#)
- [Boot Troubleshooting](#)
- [Verify VMDK Accessibility](#)

Quick Reference

- [Quick Reference: Complete Fix Procedure](#)
- [Summary of Issues and Fixes](#)

Document Information

- [Document Information](#)

Where commands run in this recovery doc. vSAN partition recovery happens in:

- **ESXi host shell (SSH)** — primary environment. `esxcli vsan ...`, `cmmds-tool`, `vsan-health-status.py`, `vsanmgmt log` inspection. Connect to each cluster member.
- **vCenter UI (browser)** — Skyline Health, vSAN Disk Management, Cluster Reconfiguration. At `https://<vcenter-fqdn>/ui/`.
- **vCenter shell (SSH to VCSA root)** — last-resort cluster reconciliation via `vsanmgmt-launcher`.

Replace any credential placeholder like `<your-password>` with values from your vault.

Environment Details

Component	Hostname	IP Address
ESXi Host 1	esxi01.lab.local	192.168.1.74
ESXi Host 2	esxi02.lab.local	192.168.1.75
ESXi Host 3	esxi03.lab.local	192.168.1.76
ESXi Host 4	esxi04.lab.local	192.168.1.82
vCenter Server	vcenter.lab.local	192.168.1.69
NSX VIP	nsx-vip.lab.local	192.168.1.70
vSAN Datastore	vsanDatastore (1)	-

Network Configuration

Network Type	Subnet	VMkernel	Purpose
Management	192.168.1.0/24	vmk0	ESXi Management
vMotion	192.168.11.0/24	vmk1	vMotion Traffic
vSAN	192.168.12.0/24	vmk2	vSAN Traffic

Issue: vSAN Cluster Partition

Symptoms

- vSAN datastore inaccessible
- vCenter VM unreachable
- Hosts unable to communicate on vSAN network
- vmkping between hosts failing with "Host is down" or "Network is unreachable"

Troubleshooting Procedures

Step 1: Verify vSAN Cluster Status

Check cluster membership and partition status:

```
# Run on ESXi host shell only (esxcli on the host directly via SSH)
esxcli vsan cluster get
```

PowerCLI equivalent (run on your Windows workstation):

```
# PowerCLI on Windows workstation – talks to vCenter, not the host directly
Set-PowerCLIConfiguration -InvalidCertificateAction Ignore -ParticipateInCEIP $false -Confirm:$false |
Out-Null
Connect-VIServer -Server vcenter.lab.local -User 'administrator@vsphere.local' -Password '<your-password>'
| Out-Null

# Cluster-level vSAN configuration (high-level: enabled, dedup/compression, encryption)
Get-Cluster | Get-VsanClusterConfiguration | Format-List *

# Per-host vSAN cluster membership (Sub-Cluster Member Count, Master / Backup / Agent state)
foreach ($h in (Get-VMHost)) {
    $esxcli = Get-EsxCli -VMHost $h -V2
    "--- $($h.Name) ---"
    $esxcli.vsan.cluster.get.Invoke() | Format-List *
}
```

Key fields to check: - **Sub-Cluster Member Count** - Should match total host count - **Local Node State** - Should be MASTER, BACKUP, or AGENT - **Local Node Health State** - Should be HEALTHY

Check vSAN health:

```
# Run on ESXi host shell only (esxcli on the host directly via SSH)
esxcli vsan health cluster list
```

PowerCLI equivalent (run on your Windows workstation):

```
# PowerCLI on Windows workstation – Get-VsanView surfaces the same health checks as the UI's vSAN Skyline
$cluster = Get-Cluster -Name vcenter-cl01
$healthSystem = Get-VsanView -Id 'VsanVcClusterHealthSystem-vsan-cluster-health-system'
$healthSystem.VsanQueryVcClusterHealthSummary($cluster.ExtensionData.MoRef, $null, $null, $true, $null,
$false, 'defaultView') |
    Format-List *
```

List unicast agents to see all cluster members:

```
# Run on ESXi host shell only (esxcli on the host directly via SSH)
```

```
esxcli vsan cluster unicastagent list
```

PowerCLI equivalent (run on your Windows workstation):

```
# PowerCLI on Windows workstation – uses Get-EsxCLI -V2 passthrough since there is no native cmdlet
foreach ($h in (Get-VMHost)) {
    $esxcli = Get-EsxCLI -VMHost $h -V2
    "--- $($h.Name) ---"
    $esxcli.vsan.cluster.unicastagent.list.Invoke() | Format-Table -AutoSize
}
```

Step 2: Verify VMkernel Adapter IP Configuration

Check vmk2 (vSAN) IP address on each host:

```
# Run on ESXi host shell only (esxcli on the host directly via SSH)
esxcli network ip interface ipv4 get -i vmk2
```

PowerCLI equivalent (run on your Windows workstation):

```
# PowerCLI on Windows workstation – talks to vCenter
Get-VMHost | ForEach-Object {
    $vmk = Get-VMHostNetworkAdapter -VMHost $_ -Name vmk2 -VMKernel
    [pscustomobject]@{
        Host      = $_.Name
        VMKernel  = $vmk.Name
        IP        = $vmk.IP
        Subnet    = $vmk.SubnetMask
        DHCP      = $vmk.DhcpEnabled
        VsanTraf  = $vmk.VsanTrafficEnabled
    }
} | Format-Table -AutoSize
```

Expected output format:

Name	IPv4 Address	IPv4 Netmask	IPv4 Broadcast	Address Type
vmk2	192.168.12.X	255.255.255.0	192.168.12.255	STATIC

Verify all hosts are on the same subnet (192.168.12.0/24 for vSAN).

Fix: Correct VMkernel IP Address

If a host has the wrong IP address:

```
# Run on ESXi host shell only (esxcli on the host directly via SSH)
esxcli network ip interface ipv4 set -i vmk2 -I 192.168.12.X -N 255.255.255.0 -t static
```

PowerCLI equivalent (run on your Windows workstation):

```
# PowerCLI on Windows workstation – replace <hostname> and <new-ip> as needed
$h = Get-VMHost -Name '<hostname>'
$vmk = Get-VMHostNetworkAdapter -VMHost $h -Name vmk2 -VMKernel
Set-VMHostNetworkAdapter -VirtualNic $vmk `
    -IP '192.168.12.X' -SubnetMask '255.255.255.0' `
    -Confirm:$false
```

Replace `192.168.12.x` with the correct IP for that host.

Step 3: Verify Physical NIC Status

List all physical NICs and their link status:

```
# Run on ESXi host shell only (esxcli on the host directly via SSH)
esxcli network nic list
```

PowerCLI equivalent (run on your Windows workstation):

```
# PowerCLI on Windows workstation
Get-VMHost | ForEach-Object {
    Get-VMHostNetworkAdapter -VMHost $_ -Physical |
        Select-Object @{n='Host';e={$_.VMHost.Name}}, Name, Mac, BitRatePerSec, FullDuplex
} | Format-Table -AutoSize
```

Check detailed NIC information:

```
# Run on ESXi host shell only (esxcli on the host directly via SSH)
esxcli network nic get -n vmnicX
```

PowerCLI equivalent (run on your Windows workstation):

```
# PowerCLI on Windows workstation – esxcli passthrough for the full driver/firmware/link details
$h = Get-VMHost -Name '<hostname>'
$esxcli = Get-EsxCLI -VMHost $h -V2
$args = $esxcli.network.nic.get.CreateArgs()
$args.nicname = 'vmnicX'
$esxcli.network.nic.get.Invoke($args) | Format-List *
```

Check NIC traffic statistics:

```
# Run on ESXi host shell only (esxcli on the host directly via SSH)
esxcli network nic stats get -n vmnicX | grep -E "Packets|Bytes"
```

PowerCLI equivalent (run on your Windows workstation):

```
# PowerCLI on Windows workstation – pull stats and filter to Packets/Bytes properties
$h = Get-VMHost -Name '<hostname>'
$esxcli = Get-EsxCLI -VMHost $h -V2
$args = $esxcli.network.nic.stats.get.CreateArgs()
$args.nicname = 'vmnicX'
$stats = $esxcli.network.nic.stats.get.Invoke($args)
$stats | Select-Object * |
    ForEach-Object { $_.PSObject.Properties } |
    Where-Object { $_.Name -match 'Packets|Bytes' } |
    Format-Table Name, Value -AutoSize
```

If "Packets sent: 0" while "Packets received" shows traffic, the NIC is not properly configured as an active uplink.

Step 4: Verify vSwitch Configuration

List all standard vSwitches and their uplinks:

```
# Run on ESXi host shell only (esxcli on the host directly via SSH)
esxcli network vswitch standard list
```

PowerCLI equivalent (run on your Windows workstation):

```
# PowerCLI on Windows workstation
Get-VMHost | ForEach-Object {
    Get-VirtualSwitch -VMHost $_ -Standard |
        Select-Object @{n='Host';e={$_.VMHost.Name}}, Name, NumPorts, Mtu,
                      @{n='Uplinks';e={ ($_.Nic) -join ', ' }}
} | Format-Table -AutoSize
```

Alternative view:

```
# Run on ESXi host shell only (esxcfg-vswitch is host-side only)
esxcfg-vswitch -l
```

List distributed virtual switches:

```
# Run on ESXi host shell only (esxcli on the host directly via SSH)
esxcli network vswitch dvs vmware list
```

PowerCLI equivalent (run on your Windows workstation):

```
# PowerCLI on Windows workstation
Get-VDSwitch | Select-Object Name, Version, Mtu, NumUplinkPorts, NumPorts |
    Format-Table -AutoSize
```

Step 5: Check vSwitch Uplink Assignment

Verify which vmnics are assigned to each vSwitch:

```
# Run on ESXi host shell only (esxcli on the host directly via SSH)
esxcli network vswitch standard list
```

PowerCLI equivalent (run on your Windows workstation):

```
# PowerCLI on Windows workstation – Nic property holds the assigned physical uplinks
Get-VMHost | ForEach-Object {
    Get-VirtualSwitch -VMHost $_ -Standard |
        Select-Object @{n='Host';e={$_.VMHost.Name}}, Name,
                      @{n='Uplinks';e={ if ($_.Nic) { $_.Nic -join ', ' } else { '(empty)' } }}
} | Format-Table -AutoSize
```

Look for the `Uplinks:` field. If empty, no physical NIC is assigned.

Fix: Add Uplink to vSwitch

Add a vmnic to a standard vSwitch:

```
# Run on ESXi host shell only (esxcli on the host directly via SSH)
esxcli network vswitch standard uplink add -u vmnicX -v vSwitch0
```

PowerCLI equivalent (run on your Windows workstation):

```
# PowerCLI on Windows workstation
$h = Get-VMHost -Name '<hostname>'
$vsw = Get-VirtualSwitch -VMHost $h -Name vSwitch0 -Standard
$pnuc = Get-VMHostNetworkAdapter -VMHost $h -Physical -Name vmnicX
Add-VirtualSwitchPhysicalNetworkAdapter -VirtualSwitch $vsw `
    -VMHostPhysicalNic $pnuc -Confirm:$false
```

Remove an uplink:

```
# Run on ESXi host shell only (esxcli on the host directly via SSH)
esxcli network vswitch standard uplink remove -u vmnicX -v vSwitch0
```

PowerCLI equivalent (run on your Windows workstation):

```
# PowerCLI on Windows workstation
$h = Get-VMHost -Name '<hostname>'
$pnuc = Get-VMHostNetworkAdapter -VMHost $h -Physical -Name vmnicX
Remove-VirtualSwitchPhysicalNetworkAdapter -VMHostPhysicalNic $pnuc -Confirm:$false
```

Step 6: Check Failover Policy (Critical)

This is often the root cause. Check if uplinks are Active, Standby, or Unused:

```
# Run on ESXi host shell only (esxcli on the host directly via SSH)
esxcli network vswitch standard policy failover get -v vSwitch0
```

PowerCLI equivalent (run on your Windows workstation):

```
# PowerCLI on Windows workstation
Get-VMHost | ForEach-Object {
    $vsw = Get-VirtualSwitch -VMHost $_ -Name vSwitch0 -Standard -ErrorAction SilentlyContinue
    if ($vsw) {
        $tp = Get-NicTeamingPolicy -VirtualSwitch $vsw
        [pscustomobject]@{
            Host      = $_.Name
            Active    = ($tp.ActiveNic -join ',')
            Standby   = ($tp.StandbyNic -join ',')
            Unused    = ($tp.UnusedNic -join ',')
            Policy    = $tp.LoadBalancingPolicy
        }
    }
} | Format-Table -AutoSize
```

Example output showing the problem:

```
Active Adapters:
Standby Adapters:
Unused Adapters: vmnic3
```

If the vmnic is listed under "Unused Adapters", it will NOT pass traffic even though it is assigned to the vSwitch.

Fix: Set Uplink as Active Adapter

```
# Run on ESXi host shell only (esxcli on the host directly via SSH)
esxcli network vswitch standard policy failover set -v vSwitch0 -a vmnicX
```

PowerCLI equivalent (run on your Windows workstation):

```
# PowerCLI on Windows workstation - promote vmnicX to Active and clear Unused
$h = Get-VMHost -Name '<hostname>'
$vsw = Get-VirtualSwitch -VMHost $h -Name vSwitch0 -Standard
Get-NicTeamingPolicy -VirtualSwitch $vsw |
    Set-NicTeamingPolicy -MakeNicActive 'vmnicX' -MakeNicUnused @() -Confirm:$false
```

Verify the fix:

```
# Run on ESXi host shell only (esxcli on the host directly via SSH)
esxcli network vswitch standard policy failover get -v vSwitch0
```

PowerCLI equivalent (run on your Windows workstation):

```
# PowerCLI on Windows workstation
$h = Get-VMHost -Name '<hostname>'
$vsw = Get-VirtualSwitch -VMHost $h -Name vSwitch0 -Standard
Get-NicTeamingPolicy -VirtualSwitch $vsw |
    Select-Object @{n='Active';e={$_.ActiveNic -join ', '}},
                  @{n='Standby';e={$_.StandbyNic -join ', '}},
                  @{n='Unused';e={$_.UnusedNic -join ', '}}
```

Expected output:

```
Active Adapters: vmnic3
Standby Adapters:
Unused Adapters:
```

Step 7: Verify VMkernel Port Configuration

Check which portgroup and portset the VMkernel adapter is connected to:

```
# Run on ESXi host shell only (esxcli on the host directly via SSH)
esxcli network ip interface list | grep -A10 vmk2
```

PowerCLI equivalent (run on your Windows workstation):

```
# PowerCLI on Windows workstation
Get-VMHost | ForEach-Object {
    $vmk = Get-VMHostNetworkAdapter -VMHost $_ -Name vmk2 -VMKernel -ErrorAction SilentlyContinue
    if ($vmk) {
        [pscustomobject]@{
            Host           = $_.Name
            VMKernel       = $vmk.Name
            PortGroup      = $vmk.PortGroupName
            Enabled        = $vmk.ManagementTrafficEnabled -or $vmk.VsanTrafficEnabled -or
            $vmk.VMotionEnabled
            VsanTrafficEna = $vmk.VsanTrafficEnabled
            MTU            = $vmk.Mtu
        }
    }
}
```

```
} | Format-Table -AutoSize
```

Key fields: - `Portset` - Should match vSwitch name (e.g., vSwitch0) - `Portgroup` - Should be the correct portgroup (e.g., vSAN) - `Enabled` - Should be true

Step 8: Test vSAN Network Connectivity

Test connectivity from one host to another using the vSAN VMkernel:

```
# Run on ESXi host shell only (vmkping has no PowerCLI parity – needs the host's TCP/IP stack)
vmkping -I vmk2 192.168.12.X
```

Test with specific packet count:

```
# Run on ESXi host shell only (vmkping has no PowerCLI parity)
vmkping -I vmk2 192.168.12.X -c 3
```

Check ARP resolution:

```
# Run on ESXi host shell only (esxcli on the host directly via SSH)
esxcli network ip neighbor list | grep 192.168.12
```

PowerCLI equivalent (run on your Windows workstation):

```
# PowerCLI on Windows workstation – esxcli passthrough; filter to the vSAN subnet client-side
$h = Get-VMHost -Name '<hostname>'
$esxcli = Get-EsxCLI -VMHost $h -V2
$esxcli.network.ip.neighbor.list.Invoke() |
    Where-Object { $_.Address -like '192.168.12.*' } |
    Format-Table Address, MACAddress, State, Interface -AutoSize
```

If ARP shows "(incomplete)", the host cannot resolve the MAC address of the target.

Step 9: Verify Routing

Check IP routing table:

```
# Run on ESXi host shell only (esxcli on the host directly via SSH)
esxcli network ip route ipv4 list
```

PowerCLI equivalent (run on your Windows workstation):

```
# PowerCLI on Windows workstation – esxcli passthrough
foreach ($h in (Get-VMHost)) {
    $esxcli = Get-EsxCLI -VMHost $h -V2
    "--- $($h.Name) ---"
    $esxcli.network.ip.route.ipv4.list.Invoke() | Format-Table -AutoSize
}
```

Ensure there is a route for the vSAN subnet through vmk2:

Step 10: Check Security Policies

Check vSwitch security policy:

```
# Run on ESXi host shell only (esxcli on the host directly via SSH)
esxcli network vswitch standard policy security get -v vSwitch0
```

PowerCLI equivalent (run on your Windows workstation):

```
# PowerCLI on Windows workstation
Get-VMHost | ForEach-Object {
    $vsw = Get-VirtualSwitch -VMHost $_ -Name vSwitch0 -Standard -ErrorAction SilentlyContinue
    if ($vsw) {
        $sp = Get-SecurityPolicy -VirtualSwitch $vsw
        [pscustomobject]@{
            Host                = $_.Name
            AllowPromiscuous    = $sp.AllowPromiscuous
            ForgedTransmits     = $sp.ForgedTransmits
            MacChanges          = $sp.MacChanges
        }
    }
} | Format-Table -AutoSize
```

Check portgroup security policy:

```
# Run on ESXi host shell only (esxcli on the host directly via SSH)
esxcli network vswitch standard portgroup policy security get -p vSAN
```

PowerCLI equivalent (run on your Windows workstation):

```
# PowerCLI on Windows workstation
Get-VMHost | ForEach-Object {
    $pg = Get-VirtualPortGroup -VMHost $_ -Name 'vSAN' -Standard -ErrorAction SilentlyContinue
    if ($pg) {
        $sp = Get-SecurityPolicy -VirtualPortGroup $pg
        [pscustomobject]@{
            Host                = $_.Name
            PortGroup           = $pg.Name
            AllowPromiscuous    = $sp.AllowPromiscuous
            ForgedTransmits     = $sp.ForgedTransmits
            MacChanges          = $sp.MacChanges
            Inherited            = $sp.AllowPromiscuousInherited
        }
    }
} | Format-Table -AutoSize
```

vCenter VM Recovery Procedures

Register VM from Datastore

Find VMX files on vSAN datastore:

```
# Run on ESXi host shell only (filesystem traversal of /vmfs/volumes)
find /vmfs/volumes/vsan* -name "*.vmx" 2>/dev/null
```

PowerCLI equivalent (run on your Windows workstation — only works while vCenter is up):

```
# PowerCLI on Windows workstation – needs vCenter; if vCenter VM is down, use the bash form on a host
$ds = Get-Datastore -Name 'vsanDatastore'
New-PSDrive -Name vsanDS -Location $ds -PSProvider VimDatastore -Root '\\' | Out-Null
Get-ChildItem -Path 'vsanDS:\' -Recurse -Filter '*.vmx' |
    Select-Object DatastoreFullPath
Remove-PSDrive vsanDS
```

List datastore contents:

```
# Run on ESXi host shell only (filesystem listing of /vmfs/volumes)
ls -la /vmfs/volumes/vsan:XXXX/
```

PowerCLI equivalent (run on your Windows workstation — only works while vCenter is up):

```
# PowerCLI on Windows workstation
$ds = Get-Datastore -Name 'vsanDatastore'
New-PSDrive -Name vsanDS -Location $ds -PSProvider VimDatastore -Root '\\' | Out-Null
Get-ChildItem -Path 'vsanDS:\' | Format-Table Name, Length, LastWriteTime -AutoSize
Remove-PSDrive vsanDS
```

Register the VM:

```
# Run on ESXi host shell only (vim-cmd is host-side; required if vCenter VM is the one being recovered)
vim-cmd solo/registervm "/vmfs/volumes/vsan:XXXX/vcenter/vcenter.vmx"
```

PowerCLI equivalent (run on your Windows workstation — connects directly to the ESXi host because vCenter is offline during recovery):

```
# PowerCLI on Windows workstation – connect to the ESXi host directly (vCenter is offline during this recovery)
Set-PowerCLIConfiguration -InvalidCertificateAction Ignore -ParticipateInCEIP $false -Confirm:$false | Out-Null
Connect-VIServer -Server 192.168.1.74 -User root -Password '<host-password>' | Out-Null

New-VM -VMFilePath '[vsanDatastore] vcenter/vcenter.vmx' -VMHost (Get-VMHost) -RunAsync
```

VM Power Operations

List all registered VMs:

```
# Run on ESXi host shell only (vim-cmd is host-side)
vim-cmd vmsvc/getallvms
```

PowerCLI equivalent (run on your Windows workstation):

```
# PowerCLI on Windows workstation – connect to vCenter or directly to a host
Get-VM | Select-Object Name, PowerState, NumCpu, MemoryGB, VMHost | Format-Table -AutoSize
```

Check VM power state:

```
# Run on ESXi host shell only (vim-cmd is host-side)
vim-cmd vmsvc/power.getstate <vmid>
```

PowerCLI equivalent (run on your Windows workstation):

```
# PowerCLI on Windows workstation
Get-VM -Name '<vm-name>' | Select-Object Name, PowerState
```

Power on VM:

```
# Run on ESXi host shell only (vim-cmd is host-side)
vim-cmd vmsvc/power.on <vmid>
```

PowerCLI equivalent (run on your Windows workstation):

```
# PowerCLI on Windows workstation
Start-VM -VM '<vm-name>' -Confirm:$false
```

Power off VM:

```
# Run on ESXi host shell only (vim-cmd is host-side)
vim-cmd vmsvc/power.off <vmid>
```

PowerCLI equivalent (run on your Windows workstation):

```
# PowerCLI on Windows workstation – Stop-VM is hard power-off; use Shutdown-VMGuest for graceful guest shutdown
Stop-VM -VM '<vm-name>' -Confirm:$false
```

Reset VM:

```
# Run on ESXi host shell only (vim-cmd is host-side)
vim-cmd vmsvc/power.reset <vmid>
```

PowerCLI equivalent (run on your Windows workstation):

```
# PowerCLI on Windows workstation
Restart-VM -VM '<vm-name>' -Confirm:$false
```

VM Configuration

Get VM devices:

```
# Run on ESXi host shell only (vim-cmd is host-side)
vim-cmd vmsvc/device.getdevices <vmid>
```

PowerCLI equivalent (run on your Windows workstation):

```
# PowerCLI on Windows workstation – covers hard disks, network adapters, CD/floppy
$vm = Get-VM -Name '<vm-name>'
$vm | Get-HardDisk | Format-Table Name, CapacityGB, Filename, StorageFormat -AutoSize
```

```
$vm | Get-NetworkAdapter | Format-Table Name, NetworkName, MacAddress, Type -AutoSize
$vm | Get-CDDrive | Format-Table Parent, IsoPath, ConnectionState -AutoSize
```

Check VM disk files:

```
# Run on ESXi host shell only (vim-cmd is host-side)
vim-cmd vmsvc/device.getdevices <vmid> | grep -i "fileName\\|label"
```

PowerCLI equivalent (run on your Windows workstation):

```
# PowerCLI on Windows workstation
Get-VM -Name '<vm-name>' | Get-HardDisk |
    Select-Object Name, Filename, CapacityGB, StorageFormat | Format-Table -AutoSize
```

Boot Troubleshooting

Set VM to enter BIOS/EFI setup on next boot:

```
# Run on ESXi host shell only (vim-cmd is host-side)
vim-cmd vmsvc/setboot.options <vmid> enterBIOSSetup=true
```

PowerCLI equivalent (run on your Windows workstation):

```
# PowerCLI on Windows workstation – flip ExtraConfig BootOptions.EnterBIOSSetup via reconfig spec
$vm = Get-VM -Name '<vm-name>'
$spec = New-Object VMware.Vim.VirtualMachineConfigSpec
$spec.BootOptions = New-Object VMware.Vim.VirtualMachineBootOptions
$spec.BootOptions.EnterBIOSSetup = $true
$vm.ExtensionData.ReconfigVM_Task($spec) | Out-Null
```

Check VM firmware type:

```
# Run on ESXi host shell only (greps the .vmx file directly in /vmfs/volumes)
grep -i firmware /vmfs/volumes/vsan:XXXX/vcenter/vcenter.vmx
```

PowerCLI equivalent (run on your Windows workstation):

```
# PowerCLI on Windows workstation – Firmware property reports 'bios' or 'efi'
Get-VM -Name '<vm-name>' |
    Select-Object Name, @{n='Firmware';e={$_.ExtensionData.Config.Firmware}}
```

Verify VMDK Accessibility

Check VMDK metadata:

```
# Run on ESXi host shell only (vmkfstools is host-side; no PowerCLI parity for descriptor metadata)
vmkfstools -D "/vmfs/volumes/vsan:XXXX/vcenter/vcenter.vmdk"
```

Quick Reference: Complete Fix Procedure

For Each ESXi Host

1. **Check current vSwitch0 uplinks:** `bash # Run on ESXi host shell only esxcli network vswitch standard list | grep Uplinks`
2. **Check failover policy:** `bash # Run on ESXi host shell only esxcli network vswitch standard policy failover get -v vSwitch0`
3. **If uplink is in Unused Adapters, set it as Active:** `bash # Run on ESXi host shell only esxcli network vswitch standard policy failover set -v vSwitch0 -a vmnicX`
4. **Verify vmk2 IP is correct:** `bash # Run on ESXi host shell only esxcli network ip interface ipv4 get -i vmk2`
5. **Test vSAN connectivity:** `bash # Run on ESXi host shell only (vmkping has no PowerCLI parity) vmkping -I vmk2 192.168.12.X`
6. **Verify vSAN cluster health:** `bash # Run on ESXi host shell only esxcli vsan cluster get esxcli vsan health cluster list`

PowerCLI consolidated equivalent (run on your Windows workstation — covers steps 1, 2, 4, 6):

```
# PowerCLI on Windows workstation – connect to vCenter, sweep all hosts
Set-PowerCLIConfiguration -InvalidCertificateAction Ignore -ParticipateInCEIP $false -Confirm:$false | Out-Null
Connect-VIServer -Server vcenter.lab.local -User 'administrator@vsphere.local' -Password '<your-password>' | Out-Null

foreach ($h in (Get-VMHost)) {
    "=====$($h.Name)===="

    # 1. vSwitch0 uplinks
    $vsw = Get-VirtualSwitch -VMHost $h -Name vSwitch0 -Standard -ErrorAction SilentlyContinue
    if ($vsw) { "Uplinks: $($vsw.Nic -join ', ')" }

    # 2. Failover policy
    if ($vsw) {
        $tp = Get-NicTeamingPolicy -VirtualSwitch $vsw
        "Active : $($tp.ActiveNic -join ', ')"
        "Standby: $($tp.StandbyNic -join ', ')"
        "Unused : $($tp.UnusedNic -join ', ')"
    }

    # 4. vmk2 IP
    $vmk = Get-VMHostNetworkAdapter -VMHost $h -Name vmk2 -VMKernel -ErrorAction SilentlyContinue
    if ($vmk) { "vmk2 : $($vmk.IP) / $($vmk.SubnetMask)" }

    # 6. vSAN cluster membership
    $esxcli = Get-EsxCli -VMHost $h -V2
    $cl = $esxcli.vsan.cluster.get.Invoke()
    "vSAN : LocalNodeState=$($cl.LocalNodeState) MemberCount=$($cl.SubClusterMemberCount)"
}
```

Summary of Issues and Fixes

Issue	Symptom	Command to Diagnose	Fix Command
No uplink on vSwitch	Uplinks: (empty)	<code>esxcli network vswitch standard list</code>	<code>esxcli network vswitch standard uplink add -u vmnicX -v vSwitch0</code>

Issue	Symptom	Command to Diagnose	Fix Command
Uplink marked as Unused	Unused Adapters: vmnicX	<code>esxcli network vswitch standard policy failover get -v vSwitch0</code>	<code>esxcli network vswitch standard policy failover set -v vSwitch0 -a vmnicX</code>
Wrong VMkernel IP	vmk2 on wrong subnet	<code>esxcli network ip interface ipv4 4 get -i vmk2</code>	<code>esxcli network ip interface ipv4 set -i vmk2 -I X.X.X.X -N 255.255.255.0 -t static</code>
vSAN partition	Member Count < total hosts	<code>esxcli vsan cluster get</code>	Fix network connectivity issues
VM not registered	VM not in inventory	<code>vim-cmd vmsvc/getallvms</code>	<code>vim-cmd solo/registervm "/path/to/vm.vmx"</code>
VM won't boot	Black screen	Check console	<code>vim-cmd vmsvc/setboot.options <vmid> enterBIOSSetup=true</code>

Related Documents

- **VCF-Windows-Update-Crash-Recovery.pdf** — Complete recovery runbook for Windows host crash, including vSAN cluster recovery, SDDC Manager / Fleet CPU soft lockups, VCF Operations cluster re-initialization, and management component restoration.

Document Information

- **Date Created:** 2026-01-15
- **Last Updated:** 2026-05-07
- **Document Version:** 1.1
- **Environment:** VMware Cloud Foundation 9.0 Lab
- **vSAN Type:** vSAN ESA (Express Storage Architecture)

Revision History

Version	Date	Notes
1.0	2026-03-15	Initial release.
1.1	2026-05-07	Option B PowerShell-sibling pass: every translation-eligible bash block now has a paste-ready PowerCLI sibling using <code>Connect-VIServer</code> , <code>Get-VMHostNetworkAdapter</code> , <code>Get-VirtualSwitch</code> , <code>Get-NicTeamingPolicy</code> , <code>Get-VM / Start-VM / Stop-VM / Restart-VM</code> , and <code>Get-ESXCLI -V2 passthrough</code> for the <code>vsan</code> namespace. ESXi-shell-only blocks (<code>vmkping</code> , <code>vmkfstools</code> , <code>partedUtil</code> , low-level <code>esxcli vsan debug / network ip neighbor</code> lookups, raw <code>find / ls / grep</code> against <code>/vmfs/volumes</code>) flagged with "Run on ESXi host shell only" comments. Top-of-doc callout added linking to <code>../POWERSHELL-TRANSLATION-REFERENCE.md</code> .